CS413 · MIDTERM · ROBIN PENA

Lambda Arcade

Three Pieces, Many Programs

Predict: what does `2 + 3 * 4` turn into when a program may use only these three pieces?

Var

`x`

a name

Lam

`(lam x. body)`

a one-input function

App

`(f a)`

apply a function to an argument

Compile extended LAMBDA to pure lambda calculus, run both on one evaluator, and check that they agree.

THE JOB

Source versus target

Everything on the left has to disappear into the three pieces on the right.

Source: extended LAMBDA (supplied AST)

- naturals `0, 1, 2, …` and `true`, `false`
- `not` · `+ - *` · `< <= > >= == !=` · `and` · `or`
- `if c then a else b` · `let x = e1 in e2`
- `pair` · `fst` · `snd`
- `fix f(x). e` (recursion)

Target: pure lambda calculus

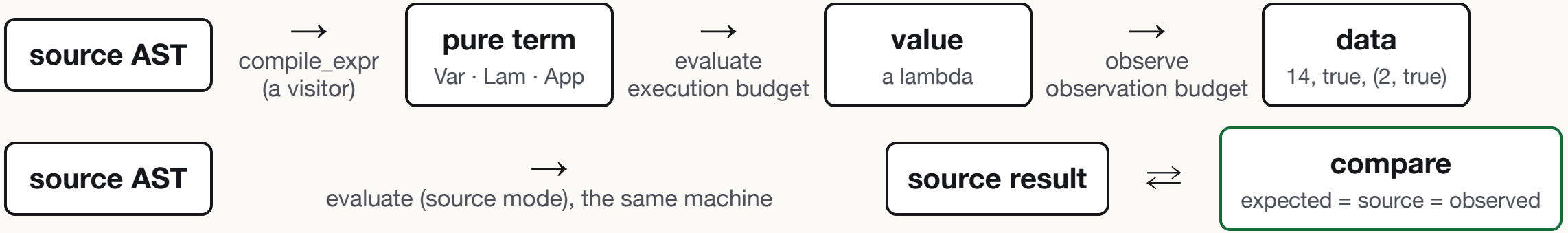
- **Var** `Var(name)`
- **Lam** `Lam(name, body)`
- **App** `App(fn, arg)`

No literals, no operators, no primitive nodes, and no leftover names like `ADD`.

Rejected at compile time, anywhere in the tree: negative literals, division, unknown operators.

ARCHITECTURE

One pipeline, one evaluator



- One machine with two small adapters: Var, Lam and App are handled the same way for both languages.
- Steps are counted in one place: the application rule.
- The compiler never imports the evaluator, and the evaluator never imports the compiler.

Data as behaviour

```
TRUE   = lam t. lam f. t
FALSE  = lam t. lam f. f
NUM(2) = lam s. lam z. s (s z)
PAIR   = lam a. lam b. lam p. p a b
FST    = lam p. p TRUE
ADD    = lam m. lam n. lam s. lam z.
           m s (n s z)
```

- A number **n** means “apply `s`, n times”.
- A boolean means “pick one of two”.
- A pair waits for a selector.
- `ADD`: do n’s applications, then m’s on top.

Same term, two meanings: `NUM(0)` and `FALSE` are both `(lam s. (lam z. z))` up to renaming, so the observer must be told which kind to expect.

SUBTRACTION

Predecessor by shifting a pair

A numeral can add an `s` but cannot take one away, so PRED rebuilds the number.

```
SHIFT (prev, cur) = (cur, cur + 1)
PRED n = first of: n shifts from (0, 0)
SUB m n = n PRED m      (apply PRED n times)
LEQ m n = ISZERO (SUB m n)
```

- `PRED 0 = 0`: no shifts happen, so the first part of `(0, 0)` comes back.
- So subtraction stops at zero: `2 − 5 = 0`.

shifts	pair	first
0	(0, 0)	0
1	(0, 1)	0
2	(1, 2)	1
3	(2, 3)	2

After k ≥ 1 shifts the pair is (k − 1, k).

NAMES

Scope, capture and fresh names

Substitute `x := y` into `(lam y. x)`:

```
wrong:  (lam y. y)      y is captured
right:  (lam y_1. y)    rename the binder
```

Renamed binders take the first unused `y_k`, so results are deterministic.

- The compiler's only new names are thunk parameters `u_k`, `v_k`, chosen to avoid every name in the program and the library.
- Nothing is reserved: a program may use `s`, `z`, `t`, `f`, `x`, even `u_1`.
- Free variables are preserved: FV(C[e]) = FV(e).
- `fix f(f). f`: the parameter wins, so `(fix f(f). f) 3` gives 3.

Real output: `if u_1 then v_1 else u_2` → `(((u_1 (lam u_3. v_1)) (lam v_2. u_2)) (lam w. w))`

TRAPDOOR

Branches become thunks

Call-by-value evaluates arguments first, so `TRUE A B` would run both A and B.

```
C[if c then a else b] = (C[c] (lam u. C[a]) (lam v. C[b])) I        I = lam w. w
C[a and b]            = (C[a] (lam u. C[b]) (lam v. FALSE)) I
C[a or b]             = (C[a] (lam u. TRUE) (lam v. C[b])) I
```

A lambda is already a value: the boolean picks one thunk, and applying it to `I` runs only that one.

Predict: `if true then 1 else LOOP`

Result: natural 1 in 3 execution steps; the LOOP thunk is dropped at step 2.

Strict code gets no thunks: `(lam x. 0) LOOP` runs LOOP first and exhausts its budget.

Why Y fails here and Z works

```
Y = lam g. (lam x. g (x x)) (lam x. g (x x))
Z = lam g. (lam x. g (lam v. x x v)) (lam x. g (lam v. x x v))
```

Y under call-by-value: `Y G` reaches `G (H H)`, where `H = lam x. G (x x)`. The argument `H H` must be evaluated first, which gives `G (H H)` again, forever. A test confirms no value appears.

Z: `lam v. x x v` is already a value, so the self-application waits until a recursive call supplies an argument.

```
C[fix f(x). e] = Z (lam f. lam x. C[e])
```

Each recursive call unrolls one more level, just as the source unrolls `fix`.

SCOREBOARD · DEMO-01

2 + 3 * 4, all the way through

source	`(2 + (3 * 4))`, source run gives 14
pure term	52 nodes: 20 Var · 13 Lam · 19 App, saved to `MySolution/demo_terms/DEMO-01.txt`
execution	completed in **4** of 10,000 beta-value steps; the value still holds the arithmetic:

```
(lam s. (lam z. (((lam s. (lam z. (s (s z)))) s)
  (((lam s. ((lam s. (lam z. (s (s (s z))))) ((lam s. (lam z. (s (s (s (s (s z)))))) s))) s) z))))
```

observation	`value STEP BASE` builds a spine of 14 nodes in **25** of 1,000 steps: natural 14, **PASS**

Execution stops at the outer lambda and never reduces inside it, so observing does the rest of the work.

```
python3 -m MySolution.demo --only DEMO-01
```

RECURSION ROUND · DEMO-10 VS DEMO-09

Factorial 4, and a loop that should stop us

`fix fact(n). if n == 0 then 1 else n * fact (n - 1) applied to 4`

	factorial 4 (DEMO-10)	(lam x. 0) LOOP (DEMO-09)
source run	24 (5 function applications)	skipped: known divergent example
pure term	344 nodes: 112 Var · 121 Lam · 111 App	35 nodes: 12 Var · 12 Lam · 11 App
execution	completed in 471 of 10,000 steps	**exhausted**: 10,000 of 10,000 used, another needed
observation	natural 24 in 218 of 1,000 steps	not attempted
result	**PASS**	**PASS**: exhaustion expected, not a proof of divergence

Also: Fibonacci 5 gives 5 in 2,021 execution and 45 observation steps.
`python3 -m MySolution.demo` → 14 of 14 examples passed.

EVIDENCE

Tests, and what they do and don’t prove

- 171 unittest tests, about 2 seconds.
- Each example checked three ways: expected = source = observed, with 1 and true kept apart.
- Budget edges: `I` at budget 0 completes in 0 steps; `I I` exhausts at 0 and completes at 1.
- Must exhaust: `let x = LOOP in 2`, `fst (pair 2 LOOP)`. Must skip: `false and LOOP`.
- 31 planted bugs: 29 caught at once; the other 2 led to a new test and a simplification.

The claim

For closed, well-typed programs with natural, boolean or pair results, observing the compiled result gives the same data as the source, given enough steps.

Why: values match by behaviour (functions: matching arguments give matching results), the evaluation order is the same, thunks match branch selection, and Z unrolls like `fix`.

A sketch, not a proof. Tests sample programs; they can’t cover all of them.

WRAP-UP

Limits, and the takeaway

- Execution needs closed programs; open programs still compile with their free variables kept.
- Unary numbers are slow: factorial 6 takes 859 execution and 6,590 observation steps; Fibonacci 10 takes 27,560 execution steps.
- Source trees nested deeper than about 330 levels hit Python’s recursion limit (the target side never recurses).
- Exhaustion is not proof of divergence, and the observers don’t catch every malformed term.

Three pieces are enough: data is behaviour, branches are thunks, recursion is Z.

```
python3 -m MySolution.demo
```